# Frontend Interview Blueprint

JavaScript & React — 300 Realistic Questions (+ LLD/HLD)

*Ace Product Company Interviews • Junior–Mid Level • 70% Coding*

by Upamanyu Deka

By Upamanyu Deka

# Frontend Interview Blueprint JavaScript & React — 300 Realistic Questions (+ LLD/HLD)

Junior–Mid Level | 70% Coding | Product-Based Companies Prepared on 23 Sep 2025

| Questions | 300 (JS/React) + 60 (Design) |
|---|---|
| Coding vs Concept | 70% vs 30% |
| Difficulty | Easy 100 • Medium 120 • Hard 80 |
| Framework | React (with core JS/DOM focus) |
| Audience | Junior → Mid-level (product companies) |

## How to Use This Book

Use this as a high-yield drill book. Start with Easy to build speed and confidence, then push into Medium and Hard. Attempt questions in a timed setting (30–45 min blocks). For coding items, write solutions in JavaScript and run simple tests. For React and system design, sketch component trees and data flow diagrams. Revisit starred questions until your approach is second nature.

### Study Plan (Suggested)

- **Week 1–2**: JS fundamentals + 30 Easy coding Qs
- **Week 3–4**: 40–50 Medium coding Qs + DOM APIs + 10 React component builds
- **Week 5–6**: 30–40 Hard/Medium coding Qs + React performance & hooks mastery
- **Week 7**: System design drills (LLD/HLD) + 6–8 design prompts
- **Week 8**: Full mocks: 1 coding + 1 React + 1 design per day

# Part I — 300 JavaScript & React Interview Questions (70% Coding)

## Easy (100)

*Warm up your fundamentals. Aim for correctness and clean code.*

## [Coding] Easy — JavaScript & DOM

1. Implement a function to check if two strings are anagrams. [Coding] [Easy] [JS]
2. Reverse a string without using built-in reverse. [Coding] [Easy] [JS]
3. Return indices of two numbers summing to target (Two Sum). [Coding] [Easy] [JS]
4. Check if a string is a palindrome; ignore non-alphanumerics. [Coding] [Easy] [JS]
5. Find the first non-repeating character in a string. [Coding] [Easy] [JS]
6. Implement Array.prototype.map (polyfill). [Coding] [Easy] [JS]
7. Implement Array.prototype.filter (polyfill). [Coding] [Easy] [JS]
8. Implement Array.prototype.reduce (polyfill). [Coding] [Easy] [JS]
9. Count character frequencies in a string (return an object). [Coding] [Easy] [JS]
10. Flatten a 1-level nested array (e.g., [1,[2,3],4] → [1,2,3,4]). [Coding] [Easy] [JS]
11. Remove duplicates from an array while preserving order. [Coding] [Easy] [JS]
12. Merge two sorted arrays into a single sorted array. [Coding] [Easy] [JS]
13. Implement debounce(fn, delay). [Coding] [Easy] [JS]
14. Implement throttle(fn, delay). [Coding] [Easy] [JS]
15. Find the longest common prefix among strings. [Coding] [Easy] [JS]
16. Capitalize the first letter of every word in a sentence. [Coding] [Easy] [JS]
17. Implement a function once(fn) that runs a function only once. [Coding] [Easy] [JS]
18. Implement a simple memoize(fn). [Coding] [Easy] [JS]
19. Chunk an array into subarrays of size k. [Coding] [Easy] [JS]
20. Group an array of objects by a key (e.g., department). [Coding] [Easy] [JS]
21. Implement deep equality check for primitives and plain objects. [Coding] [Easy] [JS]
22. Implement a curry(fn) utility. [Coding] [Easy] [JS]
23. Sum numbers in a nested array (recursively). [Coding] [Easy] [JS]
24. Implement a simple event emitter (on/off/emit). [Coding] [Easy] [JS]
25. Implement a basic Promise.all (assume inputs resolve). [Coding] [Easy] [JS]
26. Write a function to flatten an object with dot notation keys. [Coding] [Easy] [JS]
27. Implement a range(start, end) generator using closures. [Coding] [Easy] [JS]
28. Rotate an array by k positions (right rotation). [Coding] [Easy] [JS]
29. Find intersection of two arrays (unique values). [Coding] [Easy] [JS]
30. Implement a simple LRU cache (very small capacity). [Coding] [Easy] [JS]
31. Check balanced parentheses using a stack. [Coding] [Easy] [JS]
32. Implement a basic binary search on a sorted array. [Coding] [Easy] [JS]
33. Remove falsy values from an array (clean). [Coding] [Easy] [JS]
34. Sum values by key across an array of objects. [Coding] [Easy] [JS]
35. Implement get(obj, path, defaultVal) path accessor. [Coding] [Easy] [JS]
36. Implement set(obj, path, value) to create nested paths. [Coding] [Easy] [JS]
37. Write a function to dedupe objects by id. [Coding] [Easy] [JS]
38. Find the most frequent element in an array (mode). [Coding] [Easy] [JS]
39. Implement sleep(ms) returning a Promise. [Coding] [Easy] [JS]
40. Implement retry(fn, attempts) for a promise-returning function. [Coding] [Easy] [JS]
41. Serialize query params from an object (to URL string). [Coding] [Easy] [JS]
42. Parse query params into an object. [Coding] [Easy] [JS]
43. Implement a simple shuffle (Fisher–Yates). [Coding] [Easy] [JS]
44. Check if two objects have the same keys (ignore values). [Coding] [Easy] [JS]

45. Flatten a nested array to any depth (recursive). [Coding] [Easy] [JS]
46. Implement a tiny template engine: greet('Hi, ${name}!'). [Coding] [Easy] [JS]
47. Implement left-pad(str, len, ch). [Coding] [Easy] [JS]
48. Implement Promise.race (basic). [Coding] [Easy] [JS]
49. Implement a timeout wrapper for a Promise. [Coding] [Easy] [JS]
50. Implement uniqueBy(arr, keySelector). [Coding] [Easy] [JS]
51. Implement clamp(number, min, max). [Coding] [Easy] [JS]
52. Convert snake_case keys to camelCase (deep). [Coding] [Easy] [JS]
53. Implement a function to format a number with commas. [Coding] [Easy] [JS]
54. Implement memoized Fibonacci (top-down). [Coding] [Easy] [JS]
55. Implement binary to decimal conversion. [Coding] [Easy] [JS]
56. Implement decimal to binary conversion. [Coding] [Easy] [JS]
57. Check if a number is power of two. [Coding] [Easy] [JS]
58. Find missing number in 1..n given n-1 numbers. [Coding] [Easy] [JS]
59. Calculate factorial iteratively and recursively. [Coding] [Easy] [JS]
60. Implement isPrime and list primes up to n (simple). [Coding] [Easy] [JS]
61. Implement a basic hash function for strings (demo only). [Coding] [Easy] [JS]
62. Implement intersection over multiple arrays. [Coding] [Easy] [JS]
63. Implement difference(arrA, arrB). [Coding] [Easy] [JS]
64. Implement zip and unzip arrays. [Coding] [Easy] [JS]
65. Implement take and drop utilities for arrays. [Coding] [Easy] [JS]
66. Implement a safe get with optional chaining fallback. [Coding] [Easy] [JS]
67. Create a function to compare semantic versions (1.2.10 vs 1.2.9). [Coding] [Easy] [JS]
68. Implement roundTo(n, decimals). [Coding] [Easy] [JS]
69. Implement pick(obj, keys) and omit(obj, keys). [Coding] [Easy] [JS]
70. Merge unique values from multiple arrays into a sorted array. [Coding] [Easy] [JS]

## [Concept] Easy — JavaScript Theory

1. Difference between == and === with examples.
2. Explain var, let, const with temporal dead zone.
3. What is hoisting? How does it affect functions and variables?
4. Explain closures with a practical example.
5. What is the event loop? microtasks vs macrotasks?
6. Explain 'this' binding rules (default, implicit, explicit, new).
7. What are prototypes and prototypal inheritance?
8. What is a higher-order function?
9. Explain immutability and why it matters in JS/React.
10. What is debouncing vs throttling? When to use each?
11. Explain strict mode in JS.
12. What are arrow functions? lexical this?
13. What is the difference between null and undefined?
14. What are pure functions and side effects?
15. Explain DOM vs BOM.
16. How does event delegation work?
17. Explain bubbling and capturing in events.
18. What is CORS? How to handle it on the frontend?
19. What are HTTP methods and status codes (200/201/204/301/304/401/403/404/500)?
20. What is localStorage vs sessionStorage vs cookies?
21. Explain JSON.parse and JSON.stringify pitfalls (circular refs).
22. Explain Big-O at a high level for common ops in arrays/objects.
23. What is a polyfill and when is it needed?
24. Explain tree-shaking and dead code elimination.

25. What are modules (ESM) vs CommonJS?
26. Explain promises vs async/await trade-offs.
27. Explain shallow vs deep copy in JS and pitfalls.
28. Explain call stack and stack overflow in recursion.
29. What is NaN and how to check for it?
30. Explain Intl APIs (Number/Date formatting) briefly.
31. Call vs Bind vs Apply.

32. What are truthy and falsy values in JS?
33. Difference between pass-by-value and pass-by-reference in JS?
34. What is type coercion in JS? With examples.
35. What are JavaScript data types (primitive vs reference)?
36. What is IIFE (Immediately Invoked Function Expression)? Why is it used?
37. Explain default parameters and rest/spread operators.
38. What is currying in JS?
39. What is the difference between synchronous and asynchronous code?
40. What is a web worker and when should you use it?
41. Explain setTimeout vs setInterval.
42. Difference between for…in, for…of, and forEach.
43. What is destructuring in JS (array/object)?
44. Explain Map vs Set vs WeakMap vs WeakSet.
45. What is try/catch/finally in JS?
46. What are JavaScript error types (ReferenceError, TypeError, SyntaxError)?
47. What are template literals?
48. What are default imports vs named imports in ES Modules?
49. Why WeakMap keys must be objects.
50. What are generators and iterators?
51. What are Symbols and where would you actually use them?
52. How does the new.target meta-property work?
53. Explain Proxy and give a real-world use case.
54. How does Reflect API differ from Object methods?
55. What is BigInt and when should you use it?
56. How does Intl API (for dates, numbers, currencies) work?
57. What are Tagged Template Literals and their use cases?
58. How does requestIdleCallback differ from requestAnimationFrame?
59. Explain Atomics and SharedArrayBuffer at a high level.
60. What's the difference between structuredClone and JSON-based cloning?
61. How does the event loop differ between browser and Node.js?
62. What are Generator functions vs Async Generators?
63. Explain import() dynamic imports and when to use them.
64. What are Transferable Objects in postMessage (Web Workers)?

## Medium (120)

*Balance speed and depth. Focus on patterns: recursion, DP, graphs, async.*

## [Coding] Medium — Algorithms & Frontend Utilities

1. Implement deepClone handling arrays, objects, Dates, Maps, Sets, and circular refs. [Coding] [Medium] [JS]
2. Implement a basic virtual DOM diff for two trees (simplified). [Coding] [Medium] [JS]
3. Build a function to evaluate arithmetic expressions (infix) with + - * / and parentheses. [Coding] [Medium] [JS]
4. Implement an asyncPool(tasks, limit) to run promises with concurrency control. [Coding] [Medium] [JS]
5. Implement a scheduler that runs tasks at most N per second. [Coding] [Medium] [JS]
6. Implement a Least Recently Used (LRU) cache class properly. [Coding] [Medium] [JS]
7. Implement a binary heap (min-heap) with insert and extract. [Coding] [Medium] [JS]
8. Implement top-K frequent elements from an array. [Coding] [Medium] [JS]
9. Implement K-th largest element using heap/quickselect. [Coding] [Medium] [JS]
10. Flatten a deeply nested object to dot paths and unflatten back. [Coding] [Medium] [JS]
11. Implement a simple diff between two objects (added/removed/changed). [Coding] [Medium] [JS]
12. Implement a deep-equal that tolerates order-insensitive arrays of primitives. [Coding] [Medium] [JS]
13. Given nested comments, print as a flat list with depth info. [Coding] [Medium] [JS]
14. Validate a Sudoku board (rows/cols/boxes). [Coding] [Medium] [JS]
15. Implement a trie with insert/search/prefix. [Coding] [Medium] [JS]
16. Autocomplete suggestions given a dictionary with prefix search. [Coding] [Medium] [JS]
17. Implement an async memoize that caches pending promises. [Coding] [Medium] [JS]
18. Implement cancellable fetch with AbortController wrapper. [Coding] [Medium] [JS]
19. Implement retry with exponential backoff and jitter. [Coding] [Medium] [JS]
20. Implement compose and pipe utilities for functions. [Coding] [Medium] [JS]
21. Serialize and deserialize a binary tree (object structure). [Coding] [Medium] [JS]
22. Implement an observable pattern with subscribe/unsubscribe/next. [Coding] [Medium] [JS]
23. Implement a simple reactive system (dep tracking) like Vue reactivity (tiny). [Coding] [Medium] [JS]
24. Implement a pub-sub event bus with once handlers. [Coding] [Medium] [JS]
25. Implement a rate limiter (token bucket) in JS (simulation). [Coding] [Medium] [JS]
26. Given a stream of integers, return median at each step (two heaps). [Coding] [Medium] [JS]
27. Implement a basic scheduler that runs tasks in order of priority. [Coding] [Medium] [JS]
28. Implement merge intervals and insert interval problems. [Coding] [Medium] [JS]
29. Generate valid parentheses combinations for n pairs (backtracking). [Coding] [Medium] [JS]
30. Implement word break (DP) given a dictionary. [Coding] [Medium] [JS]
31. Implement cloneGraph for an undirected graph. [Coding] [Medium] [JS]
32. Implement a simple CSV parser (handle quotes and commas). [Coding] [Medium] [JS]
33. Implement a debounced auto-save editor stub. [Coding] [Medium] [JS]
34. Implement lazy evaluation chain: _(1).add(2).mul(3).value(). [Coding] [Medium] [JS]
35. Implement deep pick by predicate on object values. [Coding] [Medium] [JS]
36. Implement withTimeout(fn, ms) for any Promise-returning function. [Coding] [Medium] [JS]
37. Implement a job queue that retries failed jobs and persists state (in-memory). [Coding] [Medium] [JS]
38. Implement snake_case ■ camelCase for object keys recursively with arrays. [Coding] [Medium] [JS]
39. Implement stable sort by multiple keys with comparators. [Coding] [Medium] [JS]
40. Implement image lazy-loader using IntersectionObserver. [Coding] [Medium] [JS]
41. Implement a mini router (hash-based) to render sections. [Coding] [Medium] [JS]
42. Implement infinite scroll fetching batches, handling race conditions. [Coding] [Medium] [JS]
43. Build a drag-and-drop sortable list (logic only). [Coding] [Medium] [JS]
44. Implement a simple markdown parser for headings/lists/links. [Coding] [Medium] [JS]
45. Implement a function to diff two DOM trees and produce patch ops (simplified). [Coding] [Medium] [JS]

46. Implement a scheduler to batch DOM writes/reads to avoid layout thrash. [Coding] [Medium] [JS]
47. Implement a paginated data table with sorting and searching (logic). [Coding] [Medium] [JS]
48. Implement a promise-based semaphore/mutex utility. [Coding] [Medium] [JS]
49. Implement deep freeze for an object graph. [Coding] [Medium] [JS]
50. Implement a function to clone DOM nodes (without using cloneNode). [Coding] [Medium] [JS]
51. Implement a microtask queue shim using MutationObserver (polyfill idea). [Coding] [Medium] [JS]
52. Implement binary search tree insert/search/delete. [Coding] [Medium] [JS]
53. Implement an AVL or Red-Black tree (basic rotations). [Coding] [Medium] [JS]
54. Implement a simple event loop simulation (queues/timers). [Coding] [Medium] [JS]
55. Implement a memoized knapsack (0/1) solution. [Coding] [Medium] [JS]
56. Implement longest increasing subsequence. [Coding] [Medium] [JS]
57. Implement a function to merge k sorted linked lists. [Coding] [Medium] [JS]
58. Implement decode ways (DP) mapping digits to letters. [Coding] [Medium] [JS]
59. Implement edit distance (Levenshtein) with DP. [Coding] [Medium] [JS]
60. Implement a rolling hash (Rabin–Karp) substring search. [Coding] [Medium] [JS]
61. Implement a simple templating with conditionals/loops (mini mustache). [Coding] [Medium] [JS]
62. Implement an image carousel autoplay with pause on hover (logic). [Coding] [Medium] [JS]
63. Implement a basic scheduler for animation frames using requestAnimationFrame. [Coding] [Medium] [JS]
64. Implement a binary indexed tree (Fenwick tree) for prefix sums. [Coding] [Medium] [JS]
65. Implement a segment tree for range queries. [Coding] [Medium] [JS]
66. Implement convolution of two integer arrays (naive). [Coding] [Medium] [JS]
67. Implement a flood fill on a 2D grid. [Coding] [Medium] [JS]
68. Implement island counting in a matrix (DFS/BFS). [Coding] [Medium] [JS]
69. Implement a function to compute topological order of a DAG. [Coding] [Medium] [JS]
70. Implement a simple cron parser to schedule tasks (in-memory). [Coding] [Medium] [JS]
71. Implement promise.any and promise.allSettled polyfills. [Coding] [Medium] [JS]
72. Implement a worker-pool abstraction using web workers (API sketch). [Coding] [Medium] [JS]
73. Implement a JSON schema validator (subset). [Coding] [Medium] [JS]
74. Implement retry with circuit breaker semantics. [Coding] [Medium] [JS]
75. Implement log aggregator that groups events by time windows. [Coding] [Medium] [JS]
76. Implement a cache with TTL and auto-eviction. [Coding] [Medium] [JS]
77. Implement path normalization (resolve ., ..) for file paths. [Coding] [Medium] [JS]
78. Implement URL router with dynamic params (/:id) parsing. [Coding] [Medium] [JS]
79. Implement minimal observable store (subscribe, setState, getState). [Coding] [Medium] [JS]
80. Implement binary addition for large integers as strings. [Coding] [Medium] [JS]
81. Implement arithmetic with arbitrary precision (add/multiply strings). [Coding] [Medium] [JS]
82. Implement calendar date utilities: addDays, isLeapYear, startOfWeek. [Coding] [Medium] [JS]
83. Implement a mini i18n formatter with parameter substitution. [Coding] [Medium] [JS]
84. Implement a promise-based debounce(fn, wait) that resolves with the last invocation's result. [Coding] [Medium] [JS]

## [Concept] Medium — Web & React Internals

65. Explain event loop phases (timers, pending, idle, poll, check, close). [Concept] [Medium] [Web/React]
66. How does garbage collection work in JS (mark-and-sweep basics)? [Concept] [Medium] [Web/React]
67. Explain module bundling, code splitting, and dynamic import. [Concept] [Medium] [Web/React]
68. Describe common memory leaks in front-end and how to avoid them. [Concept] [Medium] [Web/React]
69. Explain cross-site scripting (XSS) and how to prevent it in SPAs. [Concept] [Medium] [Web/React]
70. What is CSRF and how do modern apps mitigate it? [Concept] [Medium] [Web/React]
71. How do you optimize web performance for LCP, CLS, INP? [Concept] [Medium] [Web/React]

72. Explain critical rendering path and resource prioritization. [Concept] [Medium] [Web/React]
73. Explain reflow vs repaint and how to minimize layout thrashing. [Concept] [Medium] [Web/React]
74. What is a virtual DOM? How does reconciliation work? [Concept] [Medium] [Web/React]

75. Explain React hooks rules and common pitfalls (dependency arrays). [Concept] [Medium] [Web/React]
76. What are controlled vs uncontrolled components in React? [Concept] [Medium] [Web/React]
77. Explain Context API and when to avoid prop drilling. [Concept] [Medium] [Web/React]
78. What is memoization in React (memo/useMemo/useCallback)? [Concept] [Medium] [Web/React]
79. Explain React keys: why required and problems with index as key. [Concept] [Medium] [Web/React]
80. SSR vs CSR vs ISR vs SSG – when to choose each? [Concept] [Medium] [Web/React]
81. How does hydration work in React? [Concept] [Medium] [Web/React]
82. Explain Web Workers and when to use them. [Concept] [Medium] [Web/React]
83. What is HTTP/2 and HTTP/3? How do they improve performance? [Concept] [Medium] [Web/React]
84. Explain caching headers: Cache-Control, ETag, Last-Modified. [Concept] [Medium] [Web/React]
85. What are service workers? offline caching patterns (stale-while-revalidate). [Concept] [Medium] [Web/React]
86. Explain GraphQL vs REST for front-end; trade-offs. [Concept] [Medium] [Web/React]
87. Explain SameSite cookies and auth flows on SPAs. [Concept] [Medium] [Web/React]
88. How do you design for accessibility (WCAG basics, ARIA roles)? [Concept] [Medium] [Web/React]
89. Explain security headers (CSP, X-Content-Type-Options, etc.). [Concept] [Medium] [Web/React]
90. Explain source maps and their role in debugging. [Concept] [Medium] [Web/React]
91. What are web components (custom elements, shadow DOM)? [Concept] [Medium] [Web/React]
92. Describe microfrontends: pros, cons, integration patterns. [Concept] [Medium] [Web/React]
93. Explain state normalization and selector patterns in large apps. [Concept] [Medium] [Web/React]
94. How do you approach feature flags and A/B testing on the client? [Concept] [Medium] [Web/React]
95. Explain backpressure handling in async data flows. [Concept] [Medium] [Web/React]
96. Describe idempotency and retries in front-end API calls. [Concept] [Medium] [Web/React]
97. Explain long task detection and yielding to main thread. [Concept] [Medium] [Web/React]
98. Explain image optimization pipeline for modern web. [Concept] [Medium] [Web/React]
99. Explain CSP nonce/hashes and script loading strategies. [Concept] [Medium] [Web/React]
100. Describe server-driven UI patterns and trade-offs. [Concept] [Medium] [Web/React]

## Hard (80)

*Stretch problems. Emphasize architecture, performance, and correctness under constraints.*

## [Coding] Hard — Systems & Advanced Algorithms

1. Implement a fully-featured deepClone supporting functions, symbols, Maps/Sets, Dates, RegExps, and cyclic graphs. [Coding] [Hard] [JS]
2. Implement an async task scheduler with priorities, cancellation, and pause/resume. [Coding] [Hard] [JS]
3. Implement a minimal React-like renderer (createElement, render, setState) for a tiny subset. [Coding] [Hard] [JS]
4. Implement a diff/patch algorithm for JSON trees with move/rename ops. [Coding] [Hard] [JS]
5. Implement a virtualized list (windowing) with dynamic row heights (logic only). [Coding] [Hard] [JS]
6. Implement a concurrent promise pool with backpressure and fairness guarantees. [Coding] [Hard] [JS]
7. Implement a streaming CSV parser handling multi-line quoted fields. [Coding] [Hard] [JS]
8. Implement a real-time collaborative text buffer (OT/CRDT sketch). [Coding] [Hard] [JS]
9. Implement a regex engine subset (support ., *, +, ?, [] ). [Coding] [Hard] [JS]
10. Implement a persistent immutable data structure (Trie/Vector tree). [Coding] [Hard] [JS]
11. Implement a scheduler that batches microtasks and macrotasks for UI responsiveness. [Coding] [Hard] [JS]
12. Implement a diff algorithm like Myers diff for strings (LCS-based). [Coding] [Hard] [JS]
13. Implement weighted fair-queueing for multiple async producers. [Coding] [Hard] [JS]
14. Implement WebSocket reconnection with exponential backoff and jitter. [Coding] [Hard] [JS]
15. Implement a bidirectional map with O(1) lookups both ways. [Coding] [Hard] [JS]
16. Implement an AST parser for arithmetic expressions and pretty-printer. [Coding] [Hard] [JS]
17. Implement a layout engine subset (flex-layout algorithm simplified). [Coding] [Hard] [JS]
18. Implement a CSS selector engine (subset: type, class, id, descendant). [Coding] [Hard] [JS]
19. Implement a JSONPath evaluator for querying objects. [Coding] [Hard] [JS]
20. Implement a top-down parser for a tiny template language with variables/loops. [Coding] [Hard] [JS]
21. Implement a priority search tree or interval tree (choose one). [Coding] [Hard] [JS]
22. Implement a rolling window aggregator for time-series (avg/max/min). [Coding] [Hard] [JS]
23. Implement KMP string search and Boyer–Moore variants. [Coding] [Hard] [JS]
24. Implement a general backtracking solver (N-Queens, Sudoku). [Coding] [Hard] [JS]
25. Implement a concurrent LRU with async eviction hooks. [Coding] [Hard] [JS]
26. Implement a DAG scheduler that resolves task dependencies and failures. [Coding] [Hard] [JS]
27. Implement an operational transform for text insert/delete (high level). [Coding] [Hard] [JS]
28. Implement a DOM diff that preserves caret/selection positions. [Coding] [Hard] [JS]
29. Implement an offline-first cache with sync/merge conflict resolution. [Coding] [Hard] [JS]
30. Implement a consistent hashing ring for sharding (logic). [Coding] [Hard] [JS]
31. Implement a top-N per group aggregator (streaming). [Coding] [Hard] [JS]
32. Implement a WYSIWYG editor core features (bold/italic/undo). [Coding] [Hard] [JS]
33. Implement a time-based key-value store (set/get with timestamps). [Coding] [Hard] [JS]
34. Implement LFU cache class with O(1) ops. [Coding] [Hard] [JS]
35. Implement automatic batching of state updates with microtasks. [Coding] [Hard] [JS]
36. Implement bidirectional infinite scroller (chat history) logic. [Coding] [Hard] [JS]
37. Implement a JSON diff → patch → apply pipeline with conflict detection. [Coding] [Hard] [JS]
38. Implement bitmap operations for large sparse sets. [Coding] [Hard] [JS]
39. Implement a reactive spreadsheet cell dependency graph (recalc). [Coding] [Hard] [JS]
40. Implement a multi-producer, multi-consumer queue with backpressure. [Coding] [Hard] [JS]
41. Implement a task runner reading jobs from multiple sources fairly. [Coding] [Hard] [JS]
42. Implement a generalized flatten for arbitrarily nested Maps/Arrays/Objects. [Coding] [Hard] [JS]
43. Implement a real-time presence system (heartbeats, timeouts) client logic. [Coding] [Hard] [JS]
44. Implement partial application + placeholder arguments (lodash-style). [Coding] [Hard] [JS]

45. Implement serialization of cyclic graphs to a custom format and back. [Coding] [Hard] [JS]
46. Implement UTF-8 encoder/decoder from strings to bytes and back. [Coding] [Hard] [JS]
47. Implement an efficient diff for arrays of keyed objects (min ops). [Coding] [Hard] [JS]
48. Implement a throttled search with cancellation of stale requests. [Coding] [Hard] [JS]
49. Implement a form engine with schema, validation, and conditional fields. [Coding] [Hard] [JS]
50. Implement a Mersenne Twister PRNG (or other PRNG) in JS. [Coding] [Hard] [JS]
51. Implement a workerized image processing pipeline (blur/resize) sketch. [Coding] [Hard] [JS]
52. Implement a general-purpose backoff policy library (decorators). [Coding] [Hard] [JS]
53. Implement an LZ77-like compression (toy). [Coding] [Hard] [JS]
54. Implement a JS interpreter subset (variables, arithmetic, if/while). [Coding] [Hard] [JS]
55. Implement transactional updates with rollback semantics in a store. [Coding] [Hard] [JS]
56. Implement a concurrent scheduler that preserves input order of resolved results across multiple workers. [Coding] [Hard] [JS]

## [Concept] Hard — Architecture & Scale

1. Explain React reconciliation heuristics and how keys influence diffing in depth. [Concept] [Hard] [Web/React]
2. How does React's concurrent rendering improve responsiveness? [Concept] [Hard] [Web/React]
3. Explain trade-offs of immutable data structures in UI frameworks. [Concept] [Hard] [Web/React]
4. Discuss architecture of a virtual DOM: diffing strategies and pitfalls. [Concept] [Hard] [Web/React]
5. Explain design of a scalable design system (tokens, theming, composition). [Concept] [Hard] [Web/React]
6. Discuss microfrontends integration: module federation vs iframe vs build-time. [Concept] [Hard] [Web/React]
7. Explain security implications of dynamically evaluating code on the client. [Concept] [Hard] [Web/React]
8. Discuss performance of large lists: windowing, memoization, splitting work. [Concept] [Hard] [Web/React]
9. Explain scheduler APIs (requestIdleCallback) and yielding for long tasks. [Concept] [Hard] [Web/React]
10. Discuss hydration pitfalls and server/client markup mismatches. [Concept] [Hard] [Web/React]
11. Explain Web Vitals at scale and alerting/observability on frontend. [Concept] [Hard] [Web/React]
12. Discuss CSP deployment at scale with hashes/nonces and 3rd-party scripts. [Concept] [Hard] [Web/React]
13. Explain rendering architecture of Chromium (blink, layout, paint, composite). [Concept] [Hard] [Web/React]
14. Discuss trade-offs of GraphQL client caching (normalized cache vs SWR). [Concept] [Hard] [Web/React]
15. Explain statecharts/state machines for complex UIs (XState concepts). [Concept] [Hard] [Web/React]
16. Discuss offline-first strategies and sync conflict resolution patterns. [Concept] [Hard] [Web/React]
17. Explain design of client-side search (trie, inverted index, scoring). [Concept] [Hard] [Web/React]
18. Discuss performance budgets and guardrails in CI/CD for web. [Concept] [Hard] [Web/React]
19. Explain scheduling/cooperative multitasking patterns in JS (postTask). [Concept] [Hard] [Web/React]
20. Discuss ergonomics and pitfalls of CSS-in-JS at scale. [Concept] [Hard] [Web/React]
21. Explain a11y testing automation and linting in CI. [Concept] [Hard] [Web/React]
22. Discuss approach to front-end observability: logs, metrics, RUM, tracing. [Concept] [Hard] [Web/React]
23. Explain code-splitting strategies and route-based prefetching. [Concept] [Hard] [Web/React]
24. Discuss WebAssembly use-cases in front-end and integration patterns. [Concept] [Hard] [Web/React]

# Part II — System Design (Separate from the 300)

Use these prompts for front-end LLD/HLD practice. Timebox 30–45 minutes per prompt. Structure with: Requirements → Components → State/Data → Interactions → Performance → A11y → Trade-offs.

## LLD — Low-Level Design (30 Prompts)

1. Design a reusable Modal component with focus trap, keyboard nav, and portal mounting. [LLD]
2. Design an accessible Dropdown with typeahead, multi-select, and virtualization for large lists. [LLD]
3. Design a Tabs component with keyboard navigation and ARIA roles. [LLD]
4. Design a Toast/Notification system (queueing, stacking, auto-dismiss, pause on hover). [LLD]
5. Design a Date Picker with range selection and disabled dates. [LLD]
6. Design a Data Table with sorting, filtering, pagination, and column resizing. [LLD]
7. Design a Virtualized List that supports dynamic item heights. [LLD]
8. Design an Image Gallery with lazy loading and lightbox preview. [LLD]
9. Design a Form Engine with schema-based fields, validation, and conditional visibility. [LLD]
10. Design a File Uploader with chunking, retry, and progress reporting. [LLD]
11. Design a Rich Text Editor toolbar and content model for basic formatting. [LLD]
12. Design a Markdown Editor with preview and copy as HTML. [LLD]
13. Design a Mention/Autocomplete input (e.g., @user) with async search and caching. [LLD]
14. Design a Cascading Select (Country → State → City) with dependent data loading. [LLD]
15. Design a Stepper/Wizard form with validation per step and save draft. [LLD]
16. Design a Rating component with half-stars, keyboard input, and read-only mode. [LLD]
17. Design a Collapsible Tree View for nested JSON data with indeterminate checkboxes. [LLD]
18. Design a Kanban Board (drag-and-drop) with accessible interactions. [LLD]
19. Design a Color Picker with multiple representations (HEX/RGB/HSL). [LLD]
20. Design a Responsive Navbar with multi-level menus and focus management. [LLD]
21. Design a Comment Thread UI with inline replies and optimistic updates. [LLD]
22. Design a Media Player controls UI (play/pause/seek, keyboard shortcuts). [LLD]
23. Design a Map Marker clustering UI (rendering strategies, interactions). [LLD]
24. Design a Code Diff Viewer with line highlights and inline comments. [LLD]
25. Design a Skeleton Loader system with shimmer effects and sizing presets. [LLD]
26. Design a Tooltip/Popover system with positioning and collision handling. [LLD]
27. Design a Carousel with snap points, swipe gestures, and indicators. [LLD]
28. Design a PDF Viewer with page thumbnails and zoom controls. [LLD]
29. Design a Global Search box with recent queries and result categories. [LLD]
30. Design a Notification Preferences panel with granular toggles and persistence. [LLD]

## HLD — High-Level Design (30 Prompts)

1. Design the frontend architecture for an Infinite Scroll Social Feed (web + mobile web). [HLD]
2. Design a Scalable Dashboard rendering 10k+ rows with filters and live updates. [HLD]
3. Design an E-commerce Product Listing with search, filters, personalization, and SEO. [HLD]
4. Design a Chat Application (web) with typing indicators and presence (client focus). [HLD]
5. Design a Video Streaming landing page with CDN, ABR hints, and performance budgets. [HLD]
6. Design a Docs Editor (Google Docs-lite) with collaboration (client strategy). [HLD]
7. Design an Analytics Explorer for large time-series (charts, aggregation, downsampling). [HLD]
8. Design a Booking flow (calendar, slot picking, conflict handling) for a salon. [HLD]
9. Design a Maps-like POI explorer (clustering, viewport queries, caching). [HLD]
10. Design a Real-time Notifications center with grouping, dedupe, and offline queue. [HLD]
11. Design a CMS-driven Marketing Site with SSG/ISR and localized content. [HLD]
12. Design a Multi-tenant Admin Panel with role-based access and theming. [HLD]
13. Design a Performance Monitoring RUM dashboard (Web Vitals, errors, traces). [HLD]
14. Design a Feature Flag management UI with safe-rollout controls. [HLD]

15. Design a Visual Workflow Builder (drag nodes, connect edges) for automations. [HLD]
16. Design a Server-Driven UI client rendering dynamic widgets safely. [HLD]
17. Design an Image Processing UI (upload, transformations, previews) with workers. [HLD]
18. Design a Payments Checkout front-end with PCI constraints and 3DS flows. [HLD]
19. Design an Email Builder with templates, sections, and responsive output. [HLD]
20. Design a Travel Search front-end with caching, dedupe, and retries. [HLD]
21. Design an A/B Testing experiment console (metrics, guardrails, rollbacks). [HLD]
22. Design a Personalization layer (recommendations) and caching on client. [HLD]
23. Design a Globalization/i18n strategy across a large SPA (locale data loading). [HLD]
24. Design an Offline-first Notes app with sync and conflict resolution. [HLD]
25. Design a Large-scale Forms platform (schema, validation, submission resiliency). [HLD]
26. Design a Component Library architecture with tokens and theming at scale. [HLD]
27. Design a Telemetry pipeline from browser to ingestion (privacy & sampling). [HLD]
28. Design a Real-time Collaboration cursor/selection rendering strategy. [HLD]
29. Design a News Feed with ad slots, lazy ads, and cumulative layout stability. [HLD]
30. Design a Security-sensitive Admin console (CSP, permissions, auditing). [HLD]

Good luck! Practice deliberately. Iterate quickly. Ship confidence.